

Introducción a Front y Back — Taller PIP

Esta es una **pre-lectura** para el eje Front y Back del taller. La idea es brindarte las herramientas básicas y que puedas enfocar el tiempo del taller en resolver el ejercicio, no en entender qué es una etiqueta HTML.

No hace falta memorizar nada de esto. Es más útil tenerlo leído una vez y volver a mirarlo cuando te trabes con algo puntual, que intentar aprendértelo de memoria antes de programar una sola línea.

¿Qué es "Front y Back"?

Cuando usás cualquier aplicación (una web, una app de banco, Instagram), en realidad hay **dos programas distintos** funcionando:

- El **frontend** ("front") es lo que corre en tu navegador o tu celular: lo que ves, los botones que clickeás, las animaciones. Se construye con HTML, CSS y JavaScript.
- El **backend** ("back") es un programa que corre en un servidor, en algún data center, que el front no puede ver directamente. Se encarga de guardar datos, aplicar reglas de negocio, y responder cuando el front le pide información (por ejemplo, "dame la lista de productos" o "guardá este pedido").

Front y back se hablan entre sí mandándose mensajes por internet (típicamente HTTP), como dos personas que se pasan papelitos: el front pide algo, el back responde con datos, y el front decide cómo mostrarlos.

En las sesiones "modo regular" del taller, muchos ejercicios son solo front (HTML + CSS + JS corriendo en tu navegador, sin ningún servidor de por medio, con los datos escritos directamente en el código). Esto es a propósito: te deja concentrarte en la lógica del bug sin la complejidad extra de un servidor. Cuando un ejercicio sí tenga back, el material de esa sesión te lo va a aclarar.

Cómo se arma una página web

Toda página web combina tres lenguajes, cada uno con un trabajo distinto:

Lenguaje	Responde a...	Analogía
HTML	¿Qué contenido hay?	El esqueleto y los órganos: la estructura y el contenido en bruto.

Lenguaje	Responde a...	Analogía
CSS	¿Cómo se ve?	La piel, la ropa: colores, tamaños, posición.
JavaScript	¿Cómo reacciona?	Los músculos y el sistema nervioso: qué pasa cuando interactuás con algo.

Los tres viven en archivos separados y se conectan entre sí: el HTML "linkea" al CSS y al JS, y el JS puede leer y modificar el HTML en vivo. Un ejemplo mínimo:

```

<!-- index.html -->
<!DOCTYPE html>
<html lang="es">
<head>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1 id="titulo">Hola</h1>
  <button id="boton">Cambiar saludo</button>

  <script src="script.js"></script>
</body>
</html>

```

```

/* style.css */
#titulo {
  color: darkblue;
  font-family: sans-serif;
}

```

```

// script.js
document.querySelector("#boton").addEventListener("click", () => {
  document.querySelector("#titulo").textContent = "¡Hola de nuevo!";
});

```

Con esos tres archivos ya tenés el patrón completo que vas a ver en los ejercicios del taller: HTML con la estructura, CSS con el estilo, JS conectando ambos y reaccionando a lo que hace el usuario.

HTML: estructura y contenido

HTML (HyperText Markup Language) describe el contenido de una página usando **etiquetas**. La mayoría vienen de a pares, envolviendo contenido:

```
<p>Este es un párrafo.</p>
<h1>Este es un título grande</h1>
<button>Este es un botón</button>
```

Las etiquetas pueden tener **atributos**, que agregan información extra:

```

<a href="https://ejemplo.com">Un link</a>
<input type="text" placeholder="Escribí acá">
```

Algunas etiquetas comunes que vas a encontrar todo el tiempo:

- `<div>` y `<span>`: contenedores genéricos, sin significado propio — se usan para agrupar y después estilar con CSS. `<div>` es de "bloque" (ocupa una línea entera), `<span>` es "en línea" (se mezcla con texto).
- `<h1>` a `<h6>`: títulos, de más a menos importante.
- `<ul>` / `<ol>` + `<li>`: listas (sin orden / con orden) y sus ítems.
- `<button>`: un botón clickeable.
- `<input>`: un campo para que el usuario escriba o tilde algo.
- `<a>`: un link a otra página o sección.

id y class: los dos atributos que más vas a usar

Ambos sirven para "marcar" un elemento y después encontrarlo desde CSS o JS, pero tienen una diferencia clave:

- **id**: identifica a **un único** elemento en toda la página. No debería repetirse.
- **class**: se puede poner en **muchos** elementos a la vez, para agruparlos y estilarlos/manipularlos juntos.

```
<div id="resumen"> ... </div>
<!-- solo puede haber un elemento con id="resumen" en toda la página -->

<div class="tarjeta"> ... </div>
<div class="tarjeta"> ... </div>
<!-- puede haber muchos elementos con class="tarjeta" -->
```

Vas a ver este patrón todo el tiempo en los proyectos del taller: un **id** para un elemento único (como un contador o un contenedor principal), y una **class** compartida para una lista de elementos similares (como tarjetas repetidas).

CSS: cómo se ve

CSS (Cascading Style Sheets) define el estilo visual, y se conecta al HTML a través de **selectores** que dicen "a qué elementos aplica esta regla":

```
/* A TODOS los <p> */
p {
  color: gray;
}

/* A los elementos con class="tarjeta" */
.tarjeta {
  background: white;
  border-radius: 8px;
  padding: 16px;
}

/* Al único elemento con id="resumen" */
#resumen {
  display: flex;
  gap: 12px;
}
```

Fíjate el patrón: `.` antes del nombre selecciona por clase, `#` antes del nombre selecciona por id, y escribir la etiqueta sola (sin `.` ni `#`) selecciona por tipo de elemento.

Algunas propiedades CSS que vas a ver seguido:

- `color` / `background` : color de texto / de fondo.
- `padding` / `margin` : espacio interno / espacio externo de un elemento.
- `display: flex` / `display: grid` : formas de acomodar varios elementos uno al lado del otro (o en grilla) sin tener que calcular posiciones a mano.
- `border-radius` : bordes redondeados.
- `font-size` / `font-weight` : tamaño y grosor del texto.

No hace falta saber CSS de memoria — es normal ir probando valores y mirando qué pasa, o pedirle a una IA una propiedad puntual ("¿cómo centro esto verticalmente?").

JavaScript: cómo se comporta

JavaScript es el lenguaje que le da comportamiento a la página: reacciona a clicks, calcula cosas, actualiza lo que se ve.

Variables

Guardan un valor con un nombre, para poder usarlo después:

```
const nombre = "Ana"; // no se puede reasignar después
let edad = 25;        // sí se puede reasignar
edad = 26;
```

Hoy en día se usa casi siempre `const` (si el valor no va a cambiar) o `let` (si va a cambiar). Vas a ver también la palabra `var`, una forma más vieja de declarar variables, con reglas distintas (y más confusas) sobre dónde es visible — es útil poder reconocerla, aunque no se recomienda usarla en código nuevo.

Tipos de datos básicos

```
const texto = "hola";           // string (texto)
const numero = 42;              // number
const esCerto = true;          // boolean (true/false)
const lista = [1, 2, 3];        // array (lista ordenada de valores)
const persona = {               // object (datos agrupados por nombre)
  nombre: "Ana",
  edad: 25,
};
```

Los **objetos** agrupan datos relacionados bajo un mismo nombre, y se accede a cada dato con un punto: `persona.nombre` da `"Ana"`.

Los **arrays de objetos** son el patrón de datos más común en los proyectos del taller — una lista de "cosas", cada una con sus propios datos:

```
const productos = [
  { nombre: "Taladro", precio: 15000 },
  { nombre: "Martillo", precio: 8000 },
];

productos[0].nombre; // "Taladro"
```

Funciones

Agrupar código para poder reutilizarlo, con un nombre y (opcionalmente) parámetros de entrada:

```
function saludar(nombre) {  
  return `Hola, ${nombre}!`;  
}  
  
saludar("Ana"); // "Hola, Ana!"
```

También vas a ver la forma "flecha", muy usada para funciones cortas (sobre todo dentro de otra función, como callbacks de eventos):

```
const saludar = (nombre) => `Hola, ${nombre}!`;
```

Condicionales y comparación

```
const temperatura = 30;  
  
if (temperatura > 25) {  
  console.log("Hace calor");  
} else if (temperatura > 15) {  
  console.log("Templado");  
} else {  
  console.log("Frío");  
}
```

Para comparar igualdad, usá siempre `===` (tres signos igual), no `==` (dos) — `===` compara sin hacer conversiones raras de tipo por detrás, que es la fuente de bugs confusos más común para quien recién arranca.

Recorrer arrays: `.forEach()` y `.map()`

En vez de un `for` clásico, en JS moderno es más común recorrer arrays con métodos como `.forEach()` (para hacer algo con cada elemento) o `.map()` (para transformar cada elemento en otra cosa):

```
const productos = [  
  { nombre: "Taladro", precio: 15000 },  
  { nombre: "Martillo", precio: 8000 },  
];  
  
productos.forEach((producto) => {  
  console.log(producto.nombre);  
});  
  
const nombres = productos.map((producto) => producto.nombre);  
// nombres = ["Taladro", "Martillo"]
```

Este patrón — "por cada elemento del array, hacer algo" — es el corazón de cómo se pinta una lista de datos en pantalla, que vas a ver en el punto siguiente.

El DOM: cómo JS toca el HTML en vivo

Cuando el navegador carga una página, arma una representación en memoria de todo el HTML, como un árbol de elementos — eso se llama el **DOM** (Document Object Model). JavaScript puede leer y modificar ese árbol en cualquier momento, y el navegador vuelve a dibujar la pantalla al toque.

Encontrar elementos: `querySelector`

```
const titulo = document.querySelector("h1");           // el primer <h1>
const boton = document.querySelector("#mi-boton");    // el elemento con ese id
const tarjetas = document.querySelectorAll(".tarjeta"); // TODOS los que matcheen
```

Los selectores son los mismos que usarías en CSS (por etiqueta, `.clase`, `#id`) — si sabés escribir un selector CSS, ya sabés escribir un `querySelector`.

Leer y cambiar contenido

```
titulo.textContent = "Nuevo título";           // cambia el texto
titulo.innerHTML = "Nuevo <b>título</b>";      // cambia el HTML interno (permite tags)
```

`textContent` es más seguro para texto simple; `innerHTML` se usa cuando necesitás meter HTML con formato (por ejemplo, armar una tarjeta completa a partir de datos).

Reaccionar a eventos: `addEventListener`

```
boton.addEventListener("click", () => {
  console.log("me clickearon");
});
```

El primer argumento es el tipo de evento (`"click"`, `"change"`, `"submit"`, etc.), el segundo es la función que se ejecuta cuando pasa.

Crear elementos dinámicamente

Para pintar una lista de datos (el patrón central de casi todos los proyectos del taller), se combina todo lo anterior: por cada objeto de un array, se crea un elemento HTML nuevo y se agrega a la página.

```
const tareas = [
  { texto: "Comprar pan", hecha: false },
  { texto: "Lavar los platos", hecha: true },
];

const contenedor = document.querySelector("#lista-tareas");

tareas.forEach((tarea) => {
  const item = document.createElement("li");
  item.textContent = tarea.texto;
  if (tarea.hecha) {
    item.style.textDecoration = "line-through";
  }
  contenedor.appendChild(item);
});
```

Cómo va a ser la sesión

- Trabajan en grupos, cada una con su propia sala.
- Cada grupo recibe un proyecto ya armado (HTML + CSS + JS) con **uno o más bugs intencionales** para encontrar y arreglar, más algunos **extras opcionales** para seguir practicando si les queda tiempo.
- Van a resolverlo con ayuda de una IA — el material de cada sesión trae una guía puntual de cómo pedirle ayuda bien (dándole contexto, revisando lo que devuelve, iterando en vez de repetir "arreglalo").
- El código no usa frameworks ni herramientas de build — a veces va a estar en un solo archivo, a veces repartido en varias páginas y archivos que se conectan entre sí (sin `import` / `export`, con `<script>` normales). En cualquier caso, vas a tener que leer varios archivos completos para encontrar el problema — no alcanza con mirar un fragmento aislado.

Cada sesión trae su propio material (`CONSIGNA.md`, `TEORIA.md`, `GUIA_IA.md`) con los detalles específicos de ese ejercicio puntual — esta introducción es la base común que no cambia de una sesión a otra.

Si te trabás con algo que no está acá

[MDN Web Docs](#) es la referencia más confiable y completa de HTML, CSS y JS, en español.

Buscar ahí "mdn querySelector" o "mdn addEventListener" (o lo que sea) suele ser más preciso que un tutorial random. También podés pedirle a una IA que te explique un concepto puntual — para eso también sirve, no solo para que te resuelva el bug.